

The Complete Single-Cell RNA-seq Workflow: From Raw FASTQs to Biological Insight

A full, elaborated reference covering every stage of a real scRNA-seq project — what happens, why it's necessary, and what each step actually tells you biologically. Written to be read start to finish, and returned to as reference.

THE BIG PICTURE

Single-cell RNA-seq tries to answer one core question: **"What genes are active in each individual cell, and what does that tell us about what kind of cell it is?"**

The whole workflow is really two very different jobs stacked on top of each other:

- **Upstream (Cell Ranger):** a signal-processing problem — take raw, scrambled sequencing output and correctly reconstruct "which molecule came from which cell." This is engineering, not biology.
- **Downstream (Scanpy):** a data science / biology problem — take those clean molecule counts and figure out what story they tell about cell identity, health, and function. This is interpretation, not signal recovery.

Everything below walks through both, in order, exactly as data actually flows through a real experiment.

PART 1 — UPSTREAM: FROM RAW SEQUENCER OUTPUT TO A COUNT MATRIX

The setup: what a 10x Genomics experiment physically does

Before any computation happens, here's what happened in the wet lab: thousands of individual cells were each captured in their own tiny oil droplet, together with a bead carrying millions of copies of a unique DNA barcode specific to that droplet. Inside the droplet, each cell's RNA gets tagged with that droplet's barcode, plus a random short tag (the UMI) unique to each individual RNA molecule. Then all droplets are broken open together, and everything is sequenced **in one giant pooled batch** — meaning the raw sequencer has no concept of "cells" anymore, just a soup of tagged fragments.

The entire job of upstream processing is to un-mix that soup back into per-cell, per-gene counts.

What a raw sequencing read actually contains

Every single read that comes off the sequencer is a strip of bases that encodes three stitched-together pieces of information:

Segment	What it represents	Typical length	Purpose
Cell barcode	Which droplet (cell) this came from	16 bp	Identity
UMI	A random tag unique to this one original RNA molecule	10-12 bp	Prevents double-counting from PCR
cDNA insert	The actual piece of the transcript	~90 bp	Tells you which gene

Step-by-step: what Cell Ranger does with this

Step 1 — Barcode correction

Sequencing isn't perfect; a small fraction of reads have a 1-base error somewhere in their 16bp barcode. Cell Ranger compares every observed barcode against a whitelist of ~750,000 known valid 10x barcodes and corrects near-matches back to their true value.

Why it matters: Without this, a single sequencing error would look like a brand new "cell" that never actually existed, artificially inflating your cell count with garbage.

Step 2 — Alignment to the reference genome

The ~90bp cDNA portion of each read gets mapped against the reference genome (e.g., human GRCh38) using the **STAR** aligner internally. This determines which gene, exactly, that fragment of sequence came from.

Why it matters: This is the step that converts "a random string of letters" into "this came from gene *CD3D*." Without it you have sequence, but zero biological meaning attached to it.

Step 3 — UMI counting and deduplication

This is one of the most important and most misunderstood steps, so let's go deep on it.

Before sequencing, samples go through PCR amplification — a process that exponentially copies molecules so there's enough material to physically read. A single real RNA molecule might get copied 5 times, or 80 times, entirely based on random, uneven PCR chemistry — not biology.

Without UMIs: you'd count every one of those copies as a separate molecule. A gene that happened to amplify well would look highly expressed even if it started out rare; a gene that amplified poorly would look artificially rare even if it started out abundant. Your final gene counts would reflect **PCR amplification luck**, not real biology.

With UMIs: since the random tag was attached to the molecule *before* amplification, every

PCR copy of that one original molecule shares the exact same UMI. Cell Ranger groups all reads by (cell barcode, gene, UMI) and collapses each unique combination down to a single count — regardless of how many times it was actually copied and sequenced.

Analogy: imagine writing a unique serial number on one physical letter before photocopying it 60 times. No matter how many copies exist afterward, you can always tell they all trace back to one original letter — that's exactly what a UMI lets Cell Ranger do computationally.

Why it matters: This is the difference between your final data reflecting real biological RNA abundance versus reflecting arbitrary lab noise.

Step 4 — Cell calling (separating real cells from empty droplets)

Not every droplet captured an actual cell — many capture only trace/ambient RNA floating in solution, with no real cell inside. Cell Ranger ranks every observed barcode by total UMI count and looks for a sharp drop-off point (the "knee" of the curve) that separates real, high-content cells from background noise droplets.

Why it matters: Without this, your dataset would be flooded with thousands of fake "cells" that are really just empty droplets — massively distorting every downstream statistic.

Step 5 — Final output

Cell Ranger produces a clean, filtered output:

```
filtered_feature_bc_matrix/  
├── barcodes.tsv.gz    # confirmed real cell barcodes  
├── features.tsv.gz    # gene list  
└── matrix.mtx.gz     # sparse cells x genes count matrix
```

This is precisely the file structure (`sc.read_10x_mtx()`) is built to read — this is the exact handoff point between "raw sequencing" and everything that follows in Scanpy.

A quick note on Smart-seq2 (relevant to the melanoma dataset we used)

Not every scRNA-seq method uses droplets/barcodes/UMIs at all. Older, plate-based methods like **Smart-seq2** physically sort individual cells into separate wells and sequence each well on its own — so there's no barcode demultiplexing needed, since cells were never mixed together in the first place. The tradeoff: you get far fewer total cells per experiment (hundreds to low thousands, versus tens of thousands with droplets), but often higher per-cell sequencing depth and full-length transcript coverage. The melanoma dataset we worked with (Tirosh et al., GSE72056) used this method — which is why that file arrived as pre-processed TPM values rather than a raw Cell-Ranger-style count matrix.

PART 2 — DOWNSTREAM: FROM A COUNT MATRIX TO

BIOLOGICAL MEANING

You now have (from Cell Ranger, for a real 10x experiment) a matrix: thousands of cells × thousands of genes, raw integer counts. Here's the full journey, step by step, elaborated.

Step 1 — Load the data

What: `sc.read_10x_mtx()` reads the three Cell Ranger output files into a single AnnData object in memory. **Why:** Gets raw counts into a structure Python/Scanpy tools can operate on. **Tells us:** Nothing biological yet — purely a data-loading step.

Step 2 — Quality control filtering

What: Remove cells with too few detected genes (likely empty droplets Cell Ranger's cell-calling missed), too many detected genes (likely doublets — two real cells captured in one droplet), and high mitochondrial read percentage (a signal of cell stress or death, since dying cells leak cytoplasmic mRNA while mitochondrial RNA, protected inside the mitochondria, remains relatively overrepresented).

Elaboration on doublets specifically, since this is commonly confused with "duplicates": a doublet is NOT a repeated measurement of the same cell — it's when the droplet-capture step accidentally trapped *two different cells* together, so their RNA gets tagged with the same barcode and counted as if it were one cell with an unusually large, weird combination of gene expression from two different cell types at once.

Why it matters: These filters ensure everything downstream reflects real, single, healthy cells — not technical artifacts. **Tells us:** Which cells in your dataset can be trusted for biological interpretation.

Step 3 — Normalization

What: Scale every cell's total counts to a common target sum (e.g., 10,000), then apply a $\log(x+1)$ transform. **Why:** Two cells could have identical underlying biology but end up with very different total read counts purely due to random differences in sequencing depth (how much of that cell's material happened to get sequenced). Normalization removes this technical variability so that comparisons between cells reflect biology, not sequencing luck. The log transform additionally compresses extreme outlier values so a handful of very highly-expressed genes don't dominate every later calculation. **Tells us:** Nothing biological directly — this step exists purely to make fair comparisons possible.

Step 4 — Highly variable gene (HVG) selection

What: Out of tens of thousands of genes, identify and keep only the ~2,000 whose expression varies the most across cells. **Why:** The vast majority of genes are "housekeeping" genes — expressed at similar levels in essentially every cell type, regardless of identity. They carry almost no information about what makes cells *different* from each other. The genes that vary a lot are the ones actually encoding cell-type identity

and biological state. **Tells us:** Which genes are actually worth focusing computational and interpretive effort on.

Step 5 — Scaling

What: Standardize each gene so it has mean 0 and unit variance (typically capped at a max value to control outliers). **Why:** Without this, genes with naturally larger raw expression values would dominate every downstream distance/similarity calculation, purely due to their numeric scale — not because they're more biologically important. **Tells us:** Nothing biological — pure preparation for the math that follows.

Step 6 — PCA (Principal Component Analysis)

What: Mathematically compress ~2,000 gene dimensions down to roughly 30–50 "principal components" — new combined axes that capture the maximum possible variation in the data using the fewest dimensions. **Why:** Even 2,000 genes is too many dimensions for clustering algorithms to work with efficiently and reliably — PCA finds the real underlying structure while discarding redundant, correlated noise. **Tells us:** A compact numerical representation of what makes each cell different from every other cell — but still not human-interpretable yet.

Step 7 — Neighbor graph + UMAP

What: Build a graph connecting each cell to its most similar neighbors (measured in that 30–50 dimensional PCA space), then use UMAP to project that high-dimensional graph down into a 2D layout for visualization. **Why:** The neighbor graph is the actual computational backbone that clustering will operate on. UMAP itself adds no new information — it exists purely so humans can *look* at the structure on a 2D scatterplot. **Tells us:** A visual first impression of whether distinct cell groups exist, and roughly how many — but nothing is formally decided yet.

Step 8 — Leiden clustering

What: Run the Leiden community-detection algorithm on the neighbor graph to formally assign every single cell a cluster number.

Elaboration on the resolution parameter, since this is commonly misunderstood: resolution controls how finely the algorithm splits cells into groups — think of it as a zoom level on a map, not anything about "genes vs. average."

- **Low resolution** (e.g., 0.1–0.3): zoomed out, fewer/broader clusters — might merge CD4 and CD8 T cells into one "T cell" group.
- **High resolution** (e.g., 1.0–2.0): zoomed in, more/finer clusters — might split T cells into naive, memory, and exhausted subtypes.

There's no universally correct value — it depends entirely on how granular a biological

answer your question requires, and pushing it too high risks splitting real cell types into meaningless, noise-driven fragments.

Why it matters: This turns a continuous, blurry-looking cloud of cells into discrete, analyzable groups. **Tells us:** "Cluster 0 has 1,200 cells, cluster 1 has 800 cells," etc. — still no biological names attached yet.

Step 9 — Marker gene identification

What: For every cluster, run a statistical test (commonly the Wilcoxon rank-sum test) to find which genes are significantly and distinctively higher in that cluster compared to all other cells combined. **Why:** This produces each cluster's biological "fingerprint" — the specific genes that best explain why this group of cells is different from everything else.

Tells us: The raw evidence you'll use in the next step to actually name each cluster.

Step 10 — Annotation

What: Compare each cluster's top marker genes against established biological knowledge (canonical marker gene panels) to assign real cell type labels — e.g., a cluster high in CD3D/CD3E/IL7R gets labeled "T cells"; one high in CD68/CD163 gets labeled

"Macrophages." **Why:** This is the step that requires actual domain expertise, not just running code — the algorithm found statistically distinct groups, but a knowledgeable person has to decide what those groups biologically represent. **Tells us:** Real, interpretable biology — what cell types are actually present in your sample, and in what proportions.

Step 11 — Validation

What: Sanity-check your final annotations — against published reference data, known marker panels, or (when available, as in our melanoma project) ground-truth labels from the original researchers. **Why:** Without this, you have no way to distinguish "my clustering found real biology" from "my clustering found an artifact of my specific parameter choices." This is what separates a rigorous analysis from a plausible-looking but unverified one. **Tells us:** Your actual confidence level in the final result — and if something doesn't line up, exactly where to go back and investigate.

THE FULL PIPELINE, ONE SENTENCE PER STEP

1. **FASTQ files** — raw sequencer output: barcode + UMI + gene fragment, all stitched together, no biological meaning attached yet
2. **Barcode correction** — fix small sequencing errors in cell ID tags
3. **Alignment (STAR)** — figure out which gene each fragment came from
4. **UMI deduplication** — collapse PCR-amplified copies back to true molecule counts
5. **Cell calling** — separate real captured cells from empty background droplets
6. **Output: count matrix** — clean cells × genes data, ready for analysis

7. **Load into Scanpy** — get the matrix into a workable data structure
 8. **QC filtering** — remove low-quality cells and rarely-detected genes
 9. **Normalization** — correct for differences in sequencing depth between cells
 10. **HVG selection** — focus on the genes that actually carry information
 11. **Scaling** — put all genes on equal numerical footing
 12. **PCA** — compress to the dimensions that capture real structure
 13. **Neighbors + UMAP** — find and visualize cell-to-cell similarity
 14. **Leiden clustering** — formally group similar cells together
 15. **Marker genes** — find each group's statistical biological fingerprint
 16. **Annotation** — translate fingerprints into real, named cell types
 17. **Validation** — confirm the result is real biology, not an artifact
-

GLOSSARY (quick reference)

Term	Plain-English meaning
Barcode	A short DNA tag identifying which droplet/cell a read came from
UMI	A random short tag identifying one specific original RNA molecule, used to correct for PCR overcounting
Doublet	Two real cells accidentally captured together in one droplet, counted as if it were one cell
AnnData	The Python data structure Scanpy uses to hold the cells $\times$ genes matrix plus all metadata
HVG	Highly variable genes — the subset of genes that actually differ meaningfully between cells
PCA	A dimensionality-reduction technique that compresses many gene dimensions into fewer, more informative ones
UMAP	A visualization technique that projects high-dimensional structure into 2D for human viewing
Leiden clustering	A graph-based algorithm that groups similar cells into discrete clusters
Resolution	The parameter controlling how many/how fine-grained the resulting clusters are
Marker gene	A gene whose expression is distinctively high in one particular cluster/cell type
TPM	Transcripts Per Million — a normalized expression measure common in plate-based (e.g., Smart-seq2) data